WorldForge Voxel Spawning System Integration Guide

Overview

This document explains how to integrate a voxel spawning system into WorldForge as a bolt-on visual layer. The system reads simulation state and generates voxel-based visuals without modifying simulation logic.

Core Principles

- Simulation is authoritative
- Visual system is read-only
- Deterministic seed must be preserved
- No rewrites of existing architecture
- Only hook-based integration

System Components

1. UVoxelSpawnSubsystem (WorldSubsystem)
2. SettlementVisualManager (per settlement)
3. Visual rule sets (DataAssets)
4. Spawn records for tracking visuals

Integration Steps (Phase 3 Safe)

Step 1: Create UVoxelSpawnSubsystem
Step 2: Register subsystem at world init
Step 3: Hook into ProcessorManager post-step
Step 4: Subscribe to EventBus events
Step 5: Add dirty flags to SettlementManager
Step 6: Implement spawn/update/despawn logic
Step 7: Use Instanced Static Mesh for performance

Hook Points (Minimal Changes)

- ProcessorManager: Add post-step callback
- SettlementManager: Add dirty flag setter
- EventBus: Subscribe to building/economy events
- SaveSubsystem: Optional visual state tracking

Determinism

All visual variation must use seeded RNG. No random calls allowed outside UWorldForgeRNG.

What It Creates

- Buildings (houses, markets, upgrades)
- Environment props (trees, rocks)
- Trade route visuals
- Market activity visuals
- Construction stages

Unreal Implementation

- Use ISM/HISM for repeated objects
- Use Actors for unique structures
- Use DataAssets for rules
- Use Blueprints for spawning logic

Roadmap Placement

Phase 3:
- Basic voxel spawning (buildings, props)
- Event-driven updates

Phase 4:
- Advanced city generation
- Animation syncing
- Destruction systems

Final Notes

This system enhances visual fidelity while preserving simulation integrity. It enables the player to see a living world driven entirely by underlying systems.